WEEK 8

手環、音效卡、語音小助理

藍牙把板況送手機；親手編出音效卡；板子裡住一個會查板子的語音小助理。

第一節
1 藍牙送手機
`health_ble.py`

→

第二節
2 親手編音效卡
`build-uac2.sh`

→

第三節
3 語音小助理
`voice_agent.py`

檢查點 三張證據＝Plotter、一模一樣、小助理畫面

闡門

今天從哪裡接

三件事先確認，再讓板子在背景抓編譯用的映像。

1. 在板子 `uname -r` → `7.0.0-g122c2c22d838` 不是就先照 W7 升
2. 在電腦 App Lab `0.11.0` 在板子 `arduino-app-cli version` → `Arduino App CLI version 0.14.0`
3. 在板子 `sudo apt-get install --only-upgrade -y arduino-app-cli arduino-router` →
   `0 upgraded, 0 newly installed, 0 to remove and 149 not upgraded.` 不用密碼
4. 在板子 `bash ~/w08-starter/build-uac2.sh --prefetch` 抓 headers、原始碼、映像＋編譯器；背景跑 — 2 分 16 秒

✋ 互動 **三問都舉手，才往下**

三條路

板子跟外面講話的三條路

W6 走 USB 線、W5 走 Wi-Fi；今天走藍牙低功耗（BLE）。

	USB 網路（W6）	Wi-Fi（W5）	藍牙低功耗（今天）
要線嗎	要	不用	不用
有 IP 嗎	192.168.7.2	有	？
手機直接連	不行	經過基地台	直接連

✋ 互動　猜：手機跟板子之間有沒有 IP 位址？

角色

周邊與中心：一直發名片

板子是周邊（peripheral），一直喊；手機是中心（central），掃描、挑一個連。



✋ 互動　猜：教室 30 塊板一起喊，手機掃得完嗎？

31 BYTES

名片只有 31 個位元組

傳統廣播一包最多 31 bytes；10/15 多塞發射功率就註冊失敗——名字要短、別硬塞。



服務 UUID 一段 18

廠商資料一段 11

10/15 多塞發射功率 → 註冊失敗

B0 01	03	26	02	7D 00
溫度×10	負載×10	記憶體	容器	開機分
43.2 °C	0.3	38%	2	125

示意

檢查點 手算你的溫度，跟板上 T= 比

開始喊

開始喊，找到自己

名字用組別＋座位，全班才分得出來；不准用學號、姓名。

1 在板子（starter 已在 ~/w08-starter ）

```
$ python3 ~/w08-starter/health_ble.py --name UNOQ-G03A
ready. health now: T=…
GATT application registered (6e400001-…)
advertising as 'UNOQ-G03A'
```

在手機 nRF Connect

過濾 UNOQ-G03

UNOQ-G03A RSSI 負數

UNOQ-G03B

越接近 0 越強

✓ 老師板刷成 558 實測：不用 sudo、不用裝套件，掃得到也連得上

檢查點 看到 `advertising as` 加你的名字

GATT

一棵小樹，板子主動推

連上後看到服務掛兩個特徵（characteristic）；板子用通知（notify）主動推。

6E400001 NUS 服務

- RX **6E400002** write：手機寫進來
- TX **6E400003** notify：板子推出去

RX / TX 站在板子看

看到別的服務是正常的，找 **6E400001**

一包最多 20 bytes

```
43.2,0.31,38,61,72
```

19 bytes＝1 包

```
T=43.2 L=0.31 M=38 …
```

約 40 bytes＝2 包

✋ 互動　猜：手機打字，是寫進 RX 還是 TX？

連上

連上、問它、畫它

Bluefruit Connect 連自己的板：打字問它，再開四支 yes 看溫度爬。

在手機 UART

`help` `health` `apps` `ip`

```
43.2,0.31,38,61,72
```

Plotter 五條線：溫度、負載、記憶體、硬碟、Wi-Fi

在板子

1 板上看到

```
phone subscribed (StartNotify)
RX  <- 'help'
```

在板子

2 第二個視窗

```
$ for i in 1 2 3 4; do yes > /dev/null
  & done
```

看 3 分鐘

```
$ pkill -x yes
```

看它降

檢查點 證據 1＝Plotter 溫度曲線截圖

安全

誰都連得上你的板

不用配對、不用密碼，誰都能連、能寫；所以板子只認清單上的字。

同組同學連你的板

在手機 **apps**

→ 看到你在跑什麼

✗ `os.system(text)` 手機的字直接執行

✓ `if cmd == "health":` 只認清單上的字

檢查點 說得出為什麼不能把手機的字直接執行

PART 2

第二部分：親手把音效卡編出來

板子扮 USB 音效卡（USB gadget / UAC2）要兩個 **.ko**：W7 老師給的，今天自己編、指紋要一模一樣。

1 編

`bash ~/w08-starter/build-uac2.sh`

2 分半

→

2 比指紋

`sha256sum`

四行兩兩相同

→

3 換上、重開機、驗聲音

`aplay -l`

檢查點 證據 2＝`一模一樣` 兩行＋`aplay -l`

為什麼

W7 那兩個檔從哪來

這顆核心本來沒有 USB 音訊；缺的零件，就自己編一個模組補上。

在板子

```
$ zcat /proc/config.gz | grep -E "F_UAC|G_AUDIO"
# CONFIG_USB_CONFIGFS_F_UAC1 is not set
# CONFIG_USB_CONFIGFS_F_UAC1_LEGACY is not set
# CONFIG_USB_CONFIGFS_F_UAC2 is not set
CONFIG_COMMON_CLK_AXG_AUDIO=m
```

不相干

所以：核心外面補零件

W7 老師給的： `u_audio.ko` `usb_f_uac2.ko`

＝核心模組 (kernel module) ，一個 `.ko` 檔

今天：同一套原料，自己編出同樣的兩個

檢查點 說得出 `is not set` 是什麼意思

三 樣

編模組要三樣

原料對版、工具同版，才編得出一模一樣的零件。

1 對版的核心標頭 (headers)

`apt-get download linux-headers-7.0.0-g122c2c22d838`

→ `Fetched 9,114 kB in 5s (1,901 kB/s)`

`dpkg-deb -x` 解開，裡面有 `Module.symvers`

2 對版的原始碼

5 個檔 `f_uac2.c` `u_audio.c` `u_audio.h` `u_uac2.h` `uac_common.h`

從 `arduino/linux-qcom` 的 `qcom-v7.0.0-unoq` 分支抓

3 同版編譯器

`debian:trixie` 容器（142 MB）`--prefetch`

時裝好 build-essential（75.1 MB、66 個套件）存成 `uac2-build:trixie`

`gcc (Debian 14.2.0-19) 14.2.0`

＝編核心那顆同版

檢查點 說得出三樣各從哪裡來

VERMAGIC

vermagic：為什麼要對版

每個模組都蓋了章（vermagic），寫它只配哪一版核心；差一個字就拒收。

核心（kernel）

`uname -r`

`7.0.0-g122c2c22d838`

模組（kernel module）`.ko`

`7.0.0-g122c2c22d838 SMP preempt mod_unload aarch64`

vermagic

✓ 對得上： `insmod` 收

✗ 差一個字：拒收（W7 親眼看過）

vermagic 的值來自 headers；`MODPOST` 產生 `.mod.c`、把它蓋進模組——所以 headers 要對版

檢查點 `modinfo -F vermagic` 的開頭＝ `uname -r`

動手②

編！

一支腳本：抓、解、進容器、**make**；先 **--prefetch** 過只要 24.6 秒。

在板子

```
$ bash ~/w08-starter/build-uac2.sh
warning: the compiler differs from the one used to build the kernel   正常：名字不同、版本同
CC [M]  u_audio.o
CC [M]  f_uac2.o
LD [M]  usb_f_uac2.o
MODPOST Module.symvers
LD [M]  u_audio.ko
LD [M]  usb_f_uac2.ko
vermagic:       7.0.0-g122c2c22d838 SMP preempt mod_unload aarch64
```

`real 0m24.559s` 先抓過；`real 2m29.214s` 沒先抓

`CC` ＝編譯、 `LD` ＝連結、 `MODPOST` ＝把 vermagic 蓋進模組

檢查點 看到兩行 `LD [M]` 收尾 `.ko` ＝兩個模組出來了

指紋

一模一樣

同原始碼＋同 headers＋同編譯器＝同一串位元組：可重現建置（reproducible build）。

在板子

```
$ sha256sum ~/uac2-rebuild/*.ko ~/uac2-soundcard/*.ko
6c90c70d…  /home/arduino/uac2-rebuild/u_audio.ko
5c955ddd…  /home/arduino/uac2-rebuild/usb_f_uac2.ko
6c90c70d…  /home/arduino/uac2-soundcard/u_audio.ko
5c955ddd…  /home/arduino/uac2-soundcard/usb_f_uac2.ko
```

2 腳本最後印（跟腳本裡老師的指紋比）

```
u_audio.ko   一模一樣   6c90c70d…
usb_f_uac2.ko   一模一樣   5c955ddd…
```

`u_audio.ko` 114,024 bytes

`usb_f_uac2.ko` 207,120 bytes

W7 那兩個是 2026-08-21 編的

🖐 互動　猜：原始碼改一個字，指紋會怎樣？

換上

換上你自己編的

hook 只認 `~/uac2-soundcard/`：先備份老師那份，再把你的放進去、重開機。

1 在板子 `[ -d ~/uac2-soundcard.w7 ] || cp -r ~/uac2-soundcard ~/uac2-soundcard.w7` 備份；已有就跳過

2 在板子 `cp ~/uac2-rebuild/*.ko ~/uac2-soundcard/` → `sha256sum ~/uac2-soundcard/*.ko` 還是同兩串

3 在板子 `sudo reboot` 沒密碼的人拔插線也一樣 31 秒

4 在板子 `lsmod | grep -E "u_audio|uac2"` →

```
usb_f_uac2             45056  2
u_audio                24576  2 usb_f_uac2
```

5 在板子 `cat /tmp/ncm-hook.log` → `[add] uac2 已掛` `aplay -l` → `card 1: UAC2Gadget …`

換回老師那份：`cp ~/uac2-soundcard.w7/*.ko ~/uac2-soundcard/` 再重開機

檢查點 `c.1` 四個：`acm.GS0 ffs.adb ncm.usb0 uac2.usb0`

驗收

雙向驗收

p_ 板→電腦（當麥克風）、**c_** 電腦→板（當喇叭）；兩邊都要通。

`aplay -l` → `card 1: UAC2Gadget [UAC2_Gadget], device 0: UAC2 PCM [UAC2 PCM]`

板 → 電腦 `p_srate=48000`

在電腦　麥克風 (UNO Q Audio) 開「聆聽此裝置」

在板子

```
timeout 5 speaker-test -D plughw:UAC2Gadget,0 -c 2 -t sine -f 440
```

→ 電腦聽到 440 Hz

電腦 → 板 `c_srate=64000`

在電腦　播 300 Hz 到 喇叭 (UNO Q Audio)

在板子

```
arecord -D plughw:UAC2Gadget,0 -f S16_LE -c 1 -r 16000 -d 3 /tmp/in.raw
```

→ 算峰值 `peak 32000`

電腦沒在收，板子就塞住（ aplay 卡著不動）

檢查點　證據 2 下半＝440 Hz＋`peak 32000`

PART 3

第三部分：板子裡的語音小助理

按住說話，問它板子幾度；它自己去讀板子，再用聲音回你。

1 金鑰進板子
`~/.gemini_key`

→

2 跑起來
`voice_agent.py`

→

3 開網頁、按住說話
`http://localhost:7790`

檢查點 證據 3＝畫面上那張卡：🔧 `get_board_status`

聲音的路：六段

聲音進板子、上 Gemini、再從 USB 回電腦；壞了畫面會說。

電腦
① 麥克風
聆聽此裝置
喇叭 (UNO Q Audio)

→

板子
② 錄
arecord
16 kHz 單聲道

→

Gemini
③ 送
每 0.1 秒一段
realtimeInput.audio

→

Gemini
④ 回
24 kHz PCM

→

板子
⑤ 播
aplay
plughw → 48 kHz

→

電腦
⑥ 喇叭
麥克風 (UNO Q Audio)

綠條不動＝沒進板子

沒錄到聲音：…　＝查①

聲音送不出 USB：…　＝查⑥

檢查點　每段在哪台？

即時語音：按住說話

一直開著的雙向連線（WebSocket）；聲音樣本（PCM）16 kHz 進、24 kHz 出。



`automaticActivityDetection.disabled=true`＝不讓伺服器猜你講完沒

檢查點 說得出按住、放開各送什麼

AGENT

不只是聊天

它自己決定要不要叫工具、拿到真數字才開口：agent。



✗ 只聊天：沒工具只能猜

✓ agent：自己呼叫工具 (tool calling) 再答

🖐 互動 `toolCall` 之後誰去讀板子？

動手③

金鑰、跑起來、開網頁

金鑰是密碼：貼進板子時不顯示、不留歷史。

1 在電腦 AI Studio `https://aistudio.google.com/` 建一把金鑰 現場為準

2 在板子 `read -s … → ~/.gemini_key chmod 600` 不貼進作業

3 在板子 `docker run … voice_agent.py` 整行在學生頁 p3–2 `python-apps-base:0.13.1` 或 `:0.10.1` →

語音小助理：模型 `gemini-3.8-live → …`

4 在電腦 `"$ADB" forward tcp:7790 tcp:7790 → http://localhost:7790` 大按鈕；停 `Ctrl-C` 卡住

`docker rm -f voice`

檢查點 印 `/key 是資料夾不是檔案…` = 先 `rmdir ~/.gemini_key` 再建金鑰

動手④

按住說話，問三題

問板子的事它會叫工具；問它是誰就不用。

板子現在幾度？	🔧 `get_board_status` ⟶ `52.8` 度	1.57 秒
幾點、星期幾？	🔧 `get_time` ⟶ 星期三…	1.42 秒
介紹你自己	沒叫工具	1.07 秒

放開按鈕到它開口：1.57 秒；回答長 3.73 秒；收到音量 32169；輸入 usb / 輸出 usb

檢查點 證據 3＝這張卡（工具名＋回傳 JSON）

比一比

兩條路比一比

教室機走網頁模式就能講；USB 模式在 Ubuntu 的接法沒驗。

	USB	網頁
聲音走哪	USB 音效卡	教室機攝影機的麥克風（Chrome）
放開到開口	1.57 秒	2.08 秒
電腦要設	兩個「聆聽此裝置」	`adb forward` + `localhost`

30 人同時講→按住才講、分組輪流

🖐 互動　猜：`http://192.168.7.2:7790` 為何開不了麥克風？

讀程式

讀程式：四個名字

找到四個名字，就知道它是誰、能做什麼、聲音怎麼走。

1 PERSONA

它是誰、何時該叫工具

2 TOOLS

工具清單： `get_board_status` `get_time`

3 receiver

收 Gemini 回的：聲音、逐字稿、 `toolCall` ⟶ 跑工具 ⟶ `toolResponse`

4 player_loop

把回話餵給 `aplay` ；電腦沒在收就放棄

回家挑戰 `get_wifi` ：照 `get_time` 加函式，加進 `TOOLS` `TOOL_FUNCS`

檢查點 找得到 `TOOL_FUNCS` 那一行

國立東華大學　資訊工程學系

收尾

三張證據、回家挑戰、下週

三部分各一張證據；回家替小助理多教一個工具。

1 Plotter

溫度爬上去又下來

2 指紋＋音效卡

一模一樣

`aplay -l`

3 小助理畫面

🔧 `get_board_status`

回家挑戰 `get_wifi`

清場 `pkill -x yes` `docker rm -f voice`

W9 期中展示

檢查點 Ctrl–C 看到 `bye`；小助理 `Ctrl-C` 停